

COL106 / COL1101 — Programming with Data Structures

Supplementary Practice Quiz — Set B

Format: This is a practice paper modelled directly on the department's own VPL-style programming quizzes. It is intended to be attempted the same way the real ones are: pick one question, budget roughly three hours for it, write your solution in `student_code.py` inside that question's folder, and check your own work with the provided `autograder.py` before you would ever dream of submitting anything to a real grader.

Language: Python 3, exactly as taught in lecture. **Coverage:** object-oriented programming, functional-style programming, asymptotic (worst-case and amortised) analysis, arrays and linked lists, an introduction to abstract data types and the linear ADTs (sets, lists, vectors, sequences), concurrency, and stacks and queues and their applications — i.e. everything covered up to and including the point in the course where this quiz is being given, and nothing beyond it.

A word from the instructor. Every one of these ten problems has appeared, in one form or another, in the mind of someone who has stayed up too late thinking about how a perfectly reasonable-sounding requirement ("just make it fast", "just make it thread-safe", "just don't mutate anything") turns out to demand an entire idea you have not yet had. That is the point. If you finish this paper without once staring at your own code wondering why you signed up for this degree, you did not attempt hard enough questions. Partial credit exists. Sleep, allegedly, also exists. Manage your three hours accordingly.

How each question is organised

Every question below follows the same shape as the department's real assignments: a story explaining the setting, a precise operation table defining exactly what your code must do, at least one fully worked example tracing through the operations by hand, an explicit statement of the required time complexity for every operation (this is graded, both by an automated performance smoke test AND by manual inspection of unusually clever ways to fake passing the smoke test), and a list of the files you are expected to produce. Starter code for every question is provided as `student_code.py`, with every helper method that is mechanical or bookkeeping-only already fully implemented for you, and every method that is actually the point of the question left as a stub raising `NotImplementedError` with a detailed docstring above it. Do not delete the docstrings; they are the specification.

Grading and the autograders

Each question's folder contains an `autograder.py` with roughly 15–20 test cases: two small VISIBLE cases matching the worked example and the sample files, a spread of randomised HIDDEN correctness cases across small and medium input sizes, at least one adversarial HIDDEN case specifically engineered to break a common wrong shortcut for that particular problem (e.g. a shrink-on-every-pop dynamic array, a rotation implemented by moving one element at a time n times, a BST fed already-sorted keys), and one or more large-input performance smoke tests with a generous but finite time limit, used to catch an asymptotically wrong solution (TLE) as distinct from a merely wrong one (WA) or a crashing one (RE). Every autograder prints a per-test verdict and a final mark out of 100. None of this replaces a human TA reading

your code for the complexity requirements the automated timing test cannot fully verify on its own (as several of the original department assignments this paper is modelled on explicitly warn you) — a solution that happens to scrape under the time limit by accident is not the same as a solution that is actually $O(\log n)$, and manual review exists precisely to catch the difference.

Table of contents

#	Title	Difficulty	Core idea
1	The Elastic Array	Easy-Medium	Arrays & amortised analysis
2	The Roundabout	Medium	Circular doubly linked deque
3	Persistent Grocery List	Medium	Functional immutable queue
4	The VIP Lounge Cache	Medium-Hard	Hash map + doubly linked list (LRU)
5	The Ticket-Counter Cascade	Medium-Hard	Multilevel feedback queue
6	Timeline of a Whiteboard	Hard	Persistent branching stack
7	The Overnight Bakery	Hard	Concurrency: bounded producer-consumer buffer
8	The Skyline Treap	Hard	Randomised balanced BST / order statistics
9	MinMaxQueue	Very Hard	Queue via two stacks, $O(1)$ min/max
10	The Immutable Ledger	Nightmare	Persistent trie vector + concurrent version store

Question 1: The Elastic Array

Difficulty: Easy-Medium | Points: 100 | Suggested time: 3 hours | Type of work: Individual work

Every semester some brave soul in the front row asks why we don't just use Python's built-in *list* for everything, and every semester the answer is the same: you are allowed to use it in your own code from now on, but you are not allowed to understand it by vibes. A Python list is, underneath the interpreter, a fixed-size raw block of memory (what we will call a *raw array*) together with a count of how many of its slots are actually in use. Appending past the end of that raw block cannot simply write one more word into memory that was never allocated for it — the interpreter must allocate an entirely new, larger raw block and copy every existing element into it before the new element can be added. The same is true in reverse when the list shrinks far enough that the raw block is mostly wasted space: eventually a smaller block is allocated and everything is copied down into it. Both of these events are $O(n)$ operations, and yet everyone will cheerfully tell you that `list.append` is ' $O(1)$ '. This question exists so that you can no longer say that sentence without immediately following it with the word 'amortised', and mean it.

You will build your own `DynamicArray` from scratch: no `list.append`, no `list.pop`, no `list.insert`, no slicing of your underlying storage anywhere in your solution. The only operations you may perform on your raw block are indexed reads, indexed writes, and allocating a brand-new block of a given size (a helper for this, `make_raw_array`, is given to you and simply returns a list of the requested length filled with `None` — you are to treat it exactly as if it were a fixed-capacity C array and nothing more). Your array must support `push_back` (append at the end), `pop_back` (remove and return the last element), indexed get and set, and reporting the current logical size and the current raw capacity, all while capacity is managed entirely by your own doubling and halving logic.

The push side is the easy half: whenever the raw block is completely full and a new element must be added, allocate a new block of exactly twice the old capacity, copy every live element across, and only then write the new element in. Never allocate a block that is more than double or less than double the size actually needed at that moment — a solution that grows by a fixed amount (say, always adding 64 more slots) rather than by a multiplicative factor will degrade back to $O(n)$ per push once n is large, since the number of resize events becomes proportional to n instead of $\log(n)$. The pop side is where the real content of this question lives. It is tempting to shrink the raw block the moment the logical size drops below the capacity — resist this temptation with everything you have. Consider an array sitting at exactly capacity C with C elements in it. Popping one element brings it to $C-1$, just under capacity; if you shrink immediately to capacity $C-1$ (or to $C/2$), and the very next call happens to be a push, you are instantly back at capacity and must grow again on the following push, and so on: an adversary alternating push and pop around this boundary forces a full copy on *every single call*, which is about as far from $O(1)$ amortised as it is possible to get. The standard fix, which you must reproduce and which one of the analysis questions below asks you to justify formally, is to shrink only when occupancy has dropped to one *quarter* of capacity, and even then to shrink to exactly half of capacity rather than down to the occupancy itself — this leaves enough slack that occupancy has to travel a long way (from one quarter of capacity all the way back up past capacity, or back down to a further shrink point) before another resize can possibly be triggered, and that slack is exactly what makes the amortised argument via a potential function go through. Capacity must never be allowed to fall below 1, no matter how empty the array becomes.

Operations

Command	Effect
PUSH x	push_back(x). Appends no output.
POP	pop_back(). Appends the removed value.
GET i	get(i). Appends the value at index i.
SET i x	set(i, x). Appends no output.
SIZE	Appends the current logical size.
CAP	Appends the current raw capacity.

Worked example

Starting from an empty array (capacity 1, size 0): PUSH 10 fills the single slot (capacity still 1). PUSH 20 finds the array full, so it doubles to capacity 2 before inserting; the array is now [10, 20] with capacity 2. PUSH 30 again finds the array full and doubles to capacity 4, giving [10, 20, 30] with one spare slot. Popping elements back down from size 3 towards size 1 at capacity 4 never triggers a shrink until size reaches exactly $\text{capacity}/4 = 1$, at which point capacity halves to 2. Every GET and SET in between must reflect exactly this sequence of raw-block contents; a solution that forgets to copy an element correctly during a resize will silently corrupt a later GET rather than crash outright, which is exactly the kind of bug this problem is designed to expose.

Assumptions and complexity

A single test may apply on the order of 10^5 operations. get, set, size, and capacity must be worst-case $O(1)$. push_back and pop_back must be $O(1)$ AMORTISED — individual calls that trigger a resize are of course allowed to take longer, provided the total work across any sequence of m operations starting from empty is $O(m)$. Your submission will additionally be timed against an adversarial sequence engineered specifically to punish a naive shrink-on-every-pop policy; if your average per-call cost grows with the sequence length instead of staying flat, you will see it in that test before we do.

Starter code

student_code.py gives you make_raw_array and a DynamicArray skeleton with every method stubbed to raise NotImplementedError. You must not remove or rename self._raw or self._size; you may add any other private fields.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt
- analysis.md

Question 2: The Roundabout

Difficulty: Medium | Points: 100 | Suggested time: 3 hours | Type of work: Individual work

A deque (pronounced, against all typographic sense, 'deck') is a sequence that lets you push and pop from both ends. You already know one way to build one: a doubly linked list with head and tail pointers, and a small pile of if-statements to handle the annoying case where the list is empty or has exactly one node, so that pushing or popping doesn't leave a dangling pointer somewhere. This question asks you to notice that every one of those annoying cases disappears if you simply refuse to let the list ever be 'headless': instead of head/tail pointers that can be None, keep one extra node, a *sentinel*, that never stores a real element and is simply always there. Link the list into a circle through the sentinel — `sentinel.next` is the front, `sentinel.prev` is the back, and an empty deque is simply the state where the sentinel points to itself in both directions. Every push and pop becomes a small, uniform, branch-free four-pointer rewiring, whether the deque is empty, has one element, or has a million.

Once that structure is in place, the actual point of this question is `rotate(k)`: treat the deque as sitting on a roundabout, and shift every element around by k positions, where a rotation of $+1$ moves the current front element to become the new back (as if you had called `push_back(pop_front())` once), and a rotation of -1 is its mirror image (`push_front(pop_back())`). A rotation by k , for any integer k — positive, negative, zero, or something absurd like 10^{15} — is defined as repeating that single-step primitive $|k|$ times in the corresponding direction. Doing this literally, one element at a time, costs $O(|k|)$, and the entire reason this question exists is that $|k|$ tells you nothing whatsoever about how large the deque actually is: a caller may quite reasonably hand you a deque of 40 elements and ask you to rotate it by 1,000,000,000,007. You must recognise that rotating a deque of size n by n steps (in either direction) returns it to exactly the state it started in, so only k modulo n steps of rotation are ever meaningful; and having reduced k to something in the range $[0, n)$, you must further recognise that rotating forward by k steps produces the exact same final deque as rotating backward by $n - k$ steps, so you should always perform whichever of the two is cheaper. The net effect is that your `rotate` must run in time $O(\min(k \bmod n, n - (k \bmod n)))$, never in time proportional to the raw magnitude of the k you were handed, and never by walking n elements one at a time regardless of direction chosen.

There is a second, quieter trap here for anyone who reaches for the 'obvious' implementation of moving the cheaper direction's elements one at a time via repeated push/pop calls: that is still correct and still meets the required bound, since each primitive step is $O(1)$ and you only ever take $\min(k, n-k)$ of them — but it is worth noticing, as you write it, that what you are really doing is cutting the circular list at a new point and relinking a constant number of pointers around the sentinel, and that noticing is what will make the rest of this course's balanced-tree rotations feel familiar rather than mysterious.

Operations

Command	Effect
PUSHF x	<code>push_front(x)</code> . No output.
PUSHB x	<code>push_back(x)</code> . No output.
POPF	<code>pop_front()</code> . Appends the removed value.
POPB	<code>pop_back()</code> . Appends the removed value.

Command	Effect
FRONT	Appends the current front value.
BACK	Appends the current back value.
SIZE	Appends the current size.
ROTATE k	rotate(k), for any integer k. No output.

Worked example

Building [1, 2, 3] via three PUSHB calls and then PUSHF 0 gives the deque [0, 1, 2, 3] (front to back). ROTATE 1 moves the front element (0) to the back, giving [1, 2, 3, 0]; FRONT now reports 1 and BACK reports 0. ROTATE -2 then performs the -1 primitive twice: first [0, 1, 2, 3], then [3, 0, 1, 2]; FRONT now reports 3 and BACK reports 2, and SIZE throughout remains 4, since rotation never changes how many elements are present.

Assumptions and complexity

push_front, push_back, pop_front, pop_back, front, back, and size must all be $O(1)$ worst-case. rotate(k) must be $O(\min(k \bmod n, n - (k \bmod n)))$ where n is the current size, for ANY integer k (including k with hundreds of digits, and negative k). You may not use collections.deque anywhere in this file. A single test may perform up to 10^4 deque operations interleaved with rotations whose k values are chosen specifically to be enormous compared to the deque's actual size at that moment.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 3: Persistent Grocery List

Difficulty: Medium | Points: 100 | Suggested time: 3 hours | Type of work: Individual work

A household keeps a single shared grocery list on the fridge: whoever notices milk is low adds it to the back of the list, and whoever is heading to the store takes items off the front, oldest request first. You are asked to model this list in the FUNCTIONAL style used earlier in the course for the priority queue's second part: no object is ever mutated, every operation that would 'change' the queue instead returns a brand new queue, and the queue that was passed in must remain exactly as valid and exactly as usable afterwards as it was before — including being handed to a different operation later, by different code, that has never heard of the newer queue you built from it. This is the defining feature of a *persistent* data structure, and it is not merely a stylistic preference: it is precisely what would let two different people each build their own version of tomorrow's list starting from today's shared list, without either of them being able to corrupt the other's plans.

A naive persistent queue built from a single immutable singly linked list is easy to write but forces dequeue to be $O(n)$: removing the OLDEST element from a singly linked list means removing from whichever end is expensive to reach without a backward pointer, and a purely functional list has no mutable backward pointers to exploit. The classical fix, which this question asks you to reconstruct, is to keep TWO immutable lists at once: a front list, holding the elements due to leave soonest in true queue order, and a back list, holding the most recent arrivals in REVERSE order of arrival (so that enqueue is a trivial $O(1)$ cons onto its head). Dequeuing from the front list is then also $O(1)$ — right up until the front list runs dry while the back list is not empty, at which point you must reverse the entire back list (a genuine $O(n)$ operation, and one you must write recursively, not via Python's `reversed()` or list slicing) into a fresh front list before you can serve the next dequeue. The reason this structure is still considered $O(1)$ AMORTISED, in the ordinary single-threaded use where each value is enqueued once and dequeued once, is that every individual element makes that expensive front-to-back trip at most once over its entire lifetime in the queue, no matter how many total operations you perform — the cost of the occasional expensive reversal is 'paid for' by all the enqueues that quietly built up the list being reversed.

The genuinely interesting subtlety, and the one this problem's autograder specifically probes, is what happens once persistence enters the picture in earnest: because nothing is ever mutated, a caller is free to keep an old queue reference around and call dequeue on it AGAIN later, after having already built several newer queues from it, and expect to see exactly the same oldest element it would have seen the first time. Your dequeue must honour this: dequeuing twice from the same stale queue object must produce the same first result both times, and must not corrupt or affect any other queue derived from that same stale object, including ones that were built before or after that repeated call. Implementing this correctly is mostly a matter of discipline — never assign into an existing Cell or Q, always construct and return new ones — but it is exactly the kind of discipline that is easy to break by reflex if you have spent the rest of the course writing mutating code.

Operations

Op code	Meaning	Appends to output
1 x	enqueue(x)	nothing
2	peek the oldest element	its value, or -1 if empty

Op code	Meaning	Appends to output
3	dequeue the oldest element	its value, or -1 if empty
4	how many elements are waiting	the count

Starter code

student_code.py provides the NamedTuples Cell and Q exactly as described above (the empty queue is always the literal constant `Q(EMPTY, EMPTY, 0)`, with no separate constructor to write) and a `run(state, operation)` driver identical in spirit to the one from the priority-queue assignment: it threads a new state through each operation without ever holding onto or mutating an old one itself. You must implement `enqueue`, `dequeue`, `peek_front`, `is_empty`, `size`, `to_list`, and `from_list`; the latter two must be written using recursion, not a Python loop, and `from_list` must be built on top of `enqueue` exactly the way `from_pairs` was built on top of `reduce` earlier in the course.

Assumptions and complexity

`is_empty`, `size`, and `enqueue` must be $O(1)$ worst-case. `dequeue` must be $O(1)$ amortised under the ordinary usage pattern where each queue value is consumed once along any single chain of dequeues; note that repeatedly re-dequeuing from the SAME stale queue object forces the expensive reversal to redo its work every time, which is expected and is not a bug — it is exactly the caveat that makes 'amortised under reuse' a more delicate claim than plain 'amortised', and is worth a sentence of thought before you rely on this structure inside a loop that revisits an old version. Your reversal recursion depth can be proportional to the size of the queue's back list at the moment of rebalancing; raise Python's recursion limit near the top of your file if you intend to stress-test large cases locally.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 4: The VIP Lounge Cache

Difficulty: Medium-Hard | **Points:** 100 | **Suggested time:** 3 hours | **Type of work:** Individual work

An airline lounge has room for exactly capacity guests at a time. When a guest badges in, if they were already inside, their existing spot simply gets refreshed as 'most recently seen'; if they are new and the lounge is full, the guest who has gone longest without being seen is quietly shown the door to make room. Anyone asking 'is guest k currently inside, and what is their status?' expects an answer instantly, not after the attendant walks the whole room. This is the Least Recently Used, or LRU, eviction policy, and it is one of the handful of data-structure ideas so useful that you will meet it again in an operating systems course, a databases course, and quite possibly a system-design interview, always wearing a different costume.

The constraint that makes this a genuine data-structures problem, rather than a one-line call to `functools.lru_cache` (which is, incidentally, banned in this file together with `collections.OrderedDict` — you are building the mechanism those hide, not invoking it), is that BOTH get and put must run in $O(1)$ time, and $O(1)$ time with a linked structure that must simultaneously support 'find by key instantly' and 'know who has gone longest without being touched, instantly' is only possible by keeping two structures that point directly INTO each other, rather than two separate structures that happen to agree. You will keep a dictionary mapping each key straight to a Node object, and a doubly linked list threading those SAME Node objects together in recency order, most-recently-used at the front and least-recently-used at the back. Reading key k means an $O(1)$ dictionary lookup to find its Node, followed by an $O(1)$ unlink-and-relink of that Node to the front of the list (which is exactly why it must be doubly, not singly, linked: you need to detach a node from the MIDDLE of the list in $O(1)$, which requires knowing both its neighbours). Evicting the least-recently-used guest means looking at whichever node sits at the back of the list — not searching for it, simply looking, because the list's own ordering already tells you exactly who that is.

A subtlety worth spelling out explicitly, because it is the single most common way students lose marks on this exact problem: updating the VALUE of a key that is already present must never count as inserting a new key for the purposes of deciding whether to evict. If the cache is already sitting at exactly capacity and someone calls put on a key that is already inside, you move that key to the front (it has just been 'seen') and change its value, but you do not touch capacity accounting at all and you certainly do not evict anyone — there is no new resident, so nobody needs to leave. Only a put on a genuinely new key, arriving while the cache is already full, should ever trigger an eviction.

Operations

Command	Effect
PUT k v	<code>put(k, v)</code> . No output.
GET k	<code>get(k)</code> . Appends the value, or -1 if k is not present.
SIZE	Appends the number of entries currently stored.

Input format

The first line contains two integers, capacity and q (the number of operations that follow), separated by a space. q lines follow, each one of PUT, GET, or SIZE as above.

Worked example

With capacity 2: PUT 1 100, PUT 2 200 fills the cache with 1 (least recent) and 2 (most recent). GET 1 returns 100 and makes 1 the most recent, leaving 2 as least recent. PUT 3 300 now must evict someone, and the least-recently-used entry is 2 (not 1, precisely because of the GET above), so 2 is evicted and the cache holds {1, 3}. A subsequent GET 2 correctly returns -1.

Assumptions and complexity

capacity is a fixed positive integer given once at the start and never changes. get and put must both be $O(1)$ worst-case. A single test may perform up to 2×10^5 operations against a cache whose capacity may be as small as 1 or as large as the number of distinct keys ever mentioned.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 5: The Ticket-Counter Cascade

Difficulty: Medium-Hard | **Points:** 100 | **Suggested time:** 3 hours | **Type of work:** Individual work

A government office has k service counters, numbered 0 (fastest, most in-demand) through $k-1$ (slowest, last resort), and a very particular policy for keeping the queues moving: a citizen always starts at counter 0. Counter i serves whoever is at the front of ITS line for at most $\text{base_quantum} \times 2^i$ minutes — a longer allowance at each successively slower counter, since citizens who have already been bounced down the cascade are, by that point, presumably dealing with something that genuinely needs more time. If a citizen's business is finished within their allotted time, they leave, done for good. If it is not finished, they are sent to the BACK of the NEXT counter's line (counter $i+1$) to continue later, UNLESS they were already at the very last counter, $k-1$, in which case there is nowhere further to demote them, and they simply rejoin the back of that same last counter's own line, over and over, in plain round-robin, for as long as it takes.

This is, transplanted into a government-office story, exactly the classical MULTILEVEL FEEDBACK QUEUE scheduling algorithm that real operating systems use to balance short interactive jobs against long batch jobs without needing to know in advance which is which: a process that finishes quickly (within its first, generous quantum) never has to sit behind slower processes again, while a genuinely long-running process gradually earns longer and longer uninterrupted slices, at the cost of lower priority, rather than being penalised repeatedly for its length. You are asked to build the scheduler itself: k independent FIFO queues (one per level), a way to register a newly arrived process (always into level 0's queue), and a single step operation that performs exactly one scheduling decision — find the lowest-numbered non-empty level, take the process at the front of ITS queue, run it for $\min(\text{remaining burst time, that level's quantum})$, and then either retire it (if its remaining burst hits exactly zero) or requeue it one level down (or, at the last level, back into the same level) as described above.

You are explicitly forbidden from reaching for a Python list with `pop(0)` to represent any level's queue (an $O(n)$ operation dressed up to look free), and you are equally forbidden from importing `collections.deque` for this one — you are given a fully working, from-scratch, singly-linked-list-based FIFO queue class, `SimpleQueue`, and you must build the scheduler entirely out of k instances of it. On top of the k queues, you must maintain your own $O(1)$ -lookup bookkeeping (a dictionary keyed by process id is the obvious choice) so that a STATUS query about any given process — which level is it currently sitting in, or has it already finished, or did it never exist at all — can be answered without ever having to search through any queue's contents.

Operations

Command	Effect
ARRIVE id burst	A new process id with the given total burst time arrives at level 0. No output.
RUN	Perform one scheduling step. Appends 'id DONE', 'id CONTINUE', or 'IDLE'.
STATUS id	Appends 'LEVEL I', 'DONE', or 'UNKNOWN'.
COUNT	Appends the number of processes that have arrived but not yet finished.

Input format

The first line contains three integers: num_levels, base_quantum, and q (the number of operations that follow), separated by spaces.

Assumptions and complexity

Every process id you are ever given is a distinct positive integer. Every ARRIVE, RUN, STATUS, and COUNT must run in $O(1)$ (RUN's constant factor may of course depend on num_levels, which you may assume is small, but must not depend on how many processes are currently waiting). A single test may contain on the order of 10^4 processes and 10^5 RUN calls.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 6: Timeline of a Whiteboard

Difficulty: Hard | Points: 100 | Suggested time: 3 hours | Type of work: Individual work

A design team shares one digital whiteboard that behaves like a stack of transparent sheets: someone can place a new sticky note on top, or peel the top note off, and at any instant the whole team can ask what is currently on top, or how tall the stack is. The team, however, is chaotic: rather than one linear undo history, several people independently reach back to some EARLIER recorded moment of the whiteboard and start adding or removing notes from there, creating an entire branching tree of alternate timelines, all of which must remain simultaneously valid and simultaneously queryable forever — nobody's branch is allowed to corrupt anybody else's, including branches that have not been created yet.

This is the natural continuation of the persistence idea from the Persistent Grocery List problem, pushed further in a specific direction: rather than a single thread of single-owner versions that each replace the previous one in your own code's flow of control, you must now hand out EXPLICIT, NUMBERED version identifiers, so that arbitrary external code can ask for 'version 7', build something new from it, and separately ask for 'version 7' again ten thousand operations later and get back exactly the same content, unaffected by anything that happened to versions 8 through 10006 in the meantime. Version 0 always denotes the empty stack and always will; every PUSH or POP you are asked to perform creates exactly one brand-new version, numbered sequentially in the order you process operations (1, 2, 3, ...), built from whichever existing version the operation names as its source — which need not be the most recently created version at all.

The structural technique is the same cons-cell idea used throughout the functional parts of this course: a Stack is an immutable NamedTuple pairing a (possibly EMPTY) top Cell with a cached size, and pushing x onto stack s simply builds one new Cell whose `.rest` field points at $s.top$ — every cell that made up s is reused, verbatim, unchanged, forever, by every branch that ever touches it, which is precisely what makes this $O(1)$ per operation and NOT $O(\text{size})$ despite the fact that you are conceptually handing out what looks like an entirely independent full copy of the stack every time. The autograder for this problem takes particular pleasure in building one extremely deep chain of versions and then branching off some SHALLOW, early version thousands of times in a row; if your push or your version lookup secretly walks or copies anything proportional to the size of the stack rather than doing genuinely constant work per call, this is exactly the test that will make that visible, since the total work across the deep chain plus all the shallow branches would blow up quadratically for a solution that copies, while it stays comfortably linear for one that shares structure correctly.

Operations

Command	Effect
PUSH $v\ x$	Create a new version from version v with x pushed on top. Appends the new version's id.
POP v	Create a new version from version v with its top popped. Appends ' $x\ \text{newid}$ ' (the popped value and the
TOP v	Appends the current top value of version v (does not create a new version). Assumes version v is non-
SIZE v	Appends the size of version v (does not create a new version).

Worked example

PUSH 0 1 creates version 1 = [1]. PUSH 1 2 creates version 2 = [1, 2] (branching off version 1, which remains [1] and remains valid). PUSH 0 30 creates version 3 = [30] (branching off the EMPTY version 0 again, completely independently of versions 1 and 2). At this point TOP 2 reports 2, TOP 3 reports 30, and SIZE 1 still correctly reports 1, even though two further versions have since been created elsewhere in the version graph.

Assumptions and complexity

push, pop, top, size, and looking up any version by its id must all be $O(1)$ worst-case. Nothing is ever mutated; you may not copy any existing Cell, and every operation on an existing version must leave it exactly as valid for every future operation (from this version or any other) as it was before. A single test may create on the order of 5×10^4 versions and query on the order of 10^4 old, shallow versions repeatedly.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 7: The Overnight Bakery

Difficulty: Hard | **Points:** 100 | **Suggested time:** 3 hours | **Type of work:** Individual work

Every earlier question in this paper has been about a data structure that ONE thread of control uses, one operation at a time, in some order YOU get to fully determine before you write a single line of code — even the persistent ones, where several 'branches' exist, are still each explored by a single sequential caller. That assumption ends here. A bakery runs one overnight shift with several bakers sliding fresh trays of bread onto a single display case, and several early customers lifting trays off it, ALL AT THE SAME TIME, on genuinely separate threads that the operating system may interleave in absolutely any order it pleases, at absolutely any granularity it pleases, including pausing one thread mid-sentence (mid-bytecode, really) to run another. The case itself holds at most `capacity` trays at any instant. Your job is to build the one shared object, a BoundedBuffer, that every baker and every customer thread will call into without ever being told, or needing to know, how many other threads exist or what any of them are doing right now.

Two behaviours are non-negotiable and both are about what happens at the edges of capacity. If a baker tries to add a tray while the case already holds `capacity` trays, that baker's thread must BLOCK — genuinely go to sleep, consuming no CPU while it waits — until some customer removes a tray and makes room; it must not spin in a tight while-loop repeatedly checking the size (a 'busy wait'), because that would waste an entire CPU core doing nothing useful for as long as the case stays full, and it must not simply refuse the tray or silently drop it. Symmetrically, a customer trying to take a tray from an empty case must block until some baker adds one. Getting one direction of blocking right while forgetting the other is the single most common way to fail this problem, and both directions must additionally wake the CORRECT waiting thread reliably: a tray added by a baker to a case that several customers are all waiting on must eventually be noticed by at least one of them, not lost to a missed wakeup that leaves every customer asleep forever with bread sitting untaken on the counter.

The synchronisation primitives available to you are `threading.Lock`, `threading.Condition`, and `threading.Semaphore`, in any combination you like; you may not use `queue.Queue`, `queue.SimpleQueue`, `multiprocessing`, or `asyncio`, since each of those would hand you a working bounded queue directly rather than asking you to build the mechanism underneath one. The actual storage for the waiting trays may be a plain `collections.deque` (already imported for you) — the graded difficulty here is entirely about correct synchronisation under true concurrency, not about re-implementing a FIFO queue for the fourth time this course. Because interleavings are non-deterministic, this problem cannot be graded by comparing your program's output against one fixed expected string the way every other problem in this paper is; instead, the autograder runs several small multithreaded stress programs directly against your class and checks INVARIANTS that must hold under every possible interleaving — that a `put()` on a full buffer genuinely does not return until space exists, that every item ever produced across many producer threads is consumed by EXACTLY one consumer thread (no item lost, none duplicated), and that the buffer's true occupancy never exceeds its stated capacity even under heavy concurrent load from many threads at once. A real deadlock in your solution will show up as the grading process simply hanging until it is killed by a timeout; read that as a hint about how seriously you should think through every `wait()/notify()` pairing before you submit, not as an invitation to guess and resubmit.

Interface

Method	Contract
BoundedBuffer(capacity)	Construct an empty buffer of the given fixed capacity.
put(item)	Block until there is room, then insert item at the back.
get()	Block until an item is available, then remove and return the item at the front (FIFO).
qsize()	Return the current number of items (diagnostic only; need not be perfectly synchronised).

How this is graded

There is no visible_cases text-input format for this problem (see README.txt in the starter files). The autograder directly imports BoundedBuffer and runs it under real threads with real timing, so run python3 autograder.py directly while developing rather than trying to pipe anything into it.

Assumptions and complexity

capacity is a fixed positive integer given at construction and never changes. put and get must both eventually return in $O(1)$ amortised work once the resource they are waiting on becomes available (excluding the time spent legitimately asleep waiting), and must never busy-wait. Tests may run up to 11 threads simultaneously against a single buffer instance, exchanging several thousand items.

Requested files

- student_code.py

Question 8: The Skyline Treap

Difficulty: Hard | Points: 100 | Suggested time: 3 hours | Type of work: Individual work

A city planning office maintains a live registry of every building's height. Buildings go up and come down constantly, heights repeat freely (a great many buildings share popular standard heights), and the office is asked, at any moment and just as often as it receives an update, questions like: how many buildings are exactly h metres tall? what is the height of the k -th shortest building in the entire city, counting duplicates individually? how many buildings are strictly shorter than h metres? A plain sorted Python list answers all three questions easily — and is $O(n)$ to update, every single time, because inserting or deleting from the middle of a sorted array means shifting everything after it. A plain unbalanced binary search tree fixes updates in the average case, but is only ever one adversarial (or merely unlucky, e.g. already-sorted) sequence of insertions away from degenerating into a straight line, at which point every operation becomes $O(n)$ again.

You will build a TREAP: a binary search tree ordered by key (height) in the usual way, in which every node is ALSO assigned an independent random priority, and the tree is additionally kept a max-heap on that priority via rotations. The insight that makes this worth teaching, rather than just another balanced tree with its own bespoke rebalancing casework (as you would need for an AVL or red-black tree), is that because the priorities are chosen uniformly at random and independently of insertion order, the SHAPE of the resulting tree is, with overwhelming probability, exactly the shape you would get by inserting the same keys into a plain BST in a uniformly random order — which is a classical, well-understood result to have expected depth $O(\log n)$. You get balance as a statistical consequence of randomness, essentially for free, at the cost of the guarantee being 'expected' rather than absolute worst-case (an adversary who can somehow predict or control your random number generator could still construct a bad tree, but an adversary who only controls the SEQUENCE of keys, which is the realistic threat model here, cannot).

Two further requirements complicate the classical treap just enough to make this a genuine three-hour problem rather than a copy of a textbook diagram. First, this is a MULTISSET, not a plain set: the same height may be registered by many distinct buildings, and you must track how many, without creating a separate tree node per duplicate (which would silently break the balance guarantee by letting an adversary flood one key). Second, you must support full ORDER STATISTICS — k th and rank, not merely membership — which means every node must additionally maintain the total size (with multiplicity) of its own subtree, kept up to date incrementally on every rotation and every insertion or deletion, rather than recomputed by walking the subtree, since recomputing by walking would silently turn every $O(\log n)$ operation back into $O(n)$.

Operations

Command	Effect
INSERT x	Add one occurrence of x . No output.
DELETE x	Remove one occurrence of x if present, else no-op. No output.
COUNT x	Appends how many occurrences of x are currently present.
KTH k	Appends the k -th smallest element (1-indexed, with multiplicity).
RANK x	Appends the number of elements strictly less than x .

Command	Effect
SIZE	Appends the total number of elements currently present (with multiplicity).

Worked example

INSERT 5, INSERT 2, INSERT 8, INSERT 2 leaves the multiset {2, 2, 5, 8}. COUNT 2 reports 2. KTH 1 through KTH 4 report 2, 2, 5, 8 in that order (both copies of 2 count as separate ranks). RANK 5 (elements strictly less than 5) reports 2, and RANK 1 reports 0. DELETE 2 leaves {2, 5, 8}; COUNT 2 now reports 1, and KTH 2 now reports 5.

Assumptions and complexity

insert, delete, count, kth, and rank must all be $O(\log n)$ EXPECTED, where n is the current multiset size (with multiplicity); size must be $O(1)$. You may assume kth is only ever called with $1 \leq k \leq \text{current size}$. A single test may perform on the order of 10^5 operations, including sequences of insertions given in already-sorted order specifically to check that your balance does not depend on insertion order the way a plain BST's would.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 9: MinMaxQueue

Difficulty: Very Hard | **Points:** 100 | **Suggested time:** 3 hours | **Type of work:** Individual work

Somewhere around the third week of this course, you almost certainly saw the classic party trick of implementing a stack that supports push, pop, AND `get_min`, all in $O(1)$, by keeping a second, auxiliary stack that shadows the main one and always has, at its top, the minimum of everything currently in the main stack. It is a satisfying trick precisely because a stack's discipline — you may only ever touch the most recently added element — makes 'the running minimum so far, as of each push' trivially easy to maintain incrementally: pushing x pushes $\min(x, \text{current running minimum})$ onto the shadow stack, and popping the main stack simply pops the shadow stack too. This question asks you to do the equivalent for a QUEUE, where that trick does not obviously work at all, because a queue's discipline is the opposite of a stack's: the element that leaves next is the OLDEST one, not the most recently added one, so 'the running minimum of everything currently present' can change in a complicated way the instant the oldest element (which might have BEEN the minimum) is removed, and there is no single shadow value you can maintain incrementally the way there was for the stack.

The construction you are expected to discover (or, more realistically, to have half-remembered from a systems or interview-prep context and now have to actually reconstruct correctly under pressure) combines two ideas you have already built separately, earlier in this exact course: the fact that a FIFO queue can be built from exactly TWO stacks (new arrivals are pushed onto an 'in' stack; dequeues are served from an 'out' stack, which is refilled by popping the entirety of the 'in' stack onto it, in reversed order, whenever the 'out' stack runs dry — each element makes that one-way trip at most once, which is exactly why this is $O(1)$ AMORTISED rather than $O(1)$ worst-case), and the min-tracking-stack trick from above. Put those two together, augmenting BOTH of the two stacks (not just one) with their own running minimum and maximum, and the minimum of the ENTIRE queue at any instant is simply the smaller of the two stacks' individually tracked minimums (treating an empty stack as $+\infty$ so it never wins), with the maximum handled identically on the other side. Getting the amortised argument right here is more delicate than it was for the plain two-stack queue, because now EVERY dequeue call, not just the rare ones that trigger a refill, must still answer `get_min` and `get_max` in true $O(1)$ — the amortisation only has to cover the cost of moving elements between the two stacks, never the cost of reading off a minimum or maximum, which is why each stack must maintain its OWN running extremes internally rather than you trying to recompute anything globally after a refill.

You are given a fully working MinMaxStack (push/pop/peek/peek_min/peek_max, all $O(1)$, via exactly the running-extremes-per-node trick described above) and must build MinMaxQueue entirely out of two instances of it. You may not use `collections.deque` anywhere in this file, and you may not scan the queue's contents inside `get_min` or `get_max` under any circumstances — if your `get_min` ever has to look at more than the top of each of your two stacks, you have not actually solved this problem, even if it happens to produce the right numbers.

Operations

Command	Effect
ENQ x	<code>enqueue(x)</code> . No output.
DEQ	<code>dequeue()</code> . Appends the removed value. Assumes the queue is non-empty.

Command	Effect
MIN	Appends the minimum element currently in the queue. Assumes non-empty.
MAX	Appends the maximum element currently in the queue. Assumes non-empty.
SIZE	Appends the current size.

Assumptions and complexity

enqueue and size must be $O(1)$ worst-case; get_min and get_max must be $O(1)$ worst-case, always; dequeue must be $O(1)$ amortised. A single test may perform up to 2.5×10^5 operations, including sequences of enqueue-min-max-dequeue repeated in tight succession specifically designed to force the internal transfer between your two stacks over and over.

Requested files

- student_code.py
- visible_cases_1.txt
- visible_cases_2.txt

Question 10: The Immutable Ledger

Difficulty: Nightmare | **Points:** 100 | **Suggested time:** 3 hours | **Type of work:** Individual work

This is the last question on the paper, and it is deliberately built to require everything else on it. A financial ledger records an ordered SEQUENCE of entries (an append-only log of transactions, indexable by position), and it must satisfy three requirements simultaneously that, individually, you have already solved somewhere earlier in this paper, but which have never before had to coexist inside one class. First, EVERY historical version of the ledger that has ever existed must remain readable forever, unaffected by anything appended after it — the persistence requirement from the Timeline of a Whiteboard and Persistent Grocery List problems, except now over a VECTOR (indexable by position) rather than a stack or a queue, which means a single linear chain of cons cells will no longer do: you need a genuine tree. Second, appending, reading, or overwriting an element at a given position must cost $O(\log n)$, not $O(n)$ or $O(1)$ -with-a-hidden- $O(n)$ -copy — you may not, under any circumstances, copy an entire version's worth of data just to record one small change, the way a naive 'persistent array via tuple concatenation' would. Third, and this is the part that is genuinely new, the shared store of ledger versions must be safely usable by MANY THREADS AT ONCE, exactly as the Overnight Bakery's buffer had to be, with several threads simultaneously creating brand new versions while several other threads simultaneously read arbitrary older versions, with no lock ever protecting more of the work than it strictly has to.

Part A of this problem is a from-scratch, simplified version of the persistent-vector technique used in production functional-programming runtimes (Clojure's core vector type being the most famous example, though yours will use a plain binary trie rather than its 32-way one, for sanity's sake): a PVector of size n is a complete binary trie of height h , holding up to 2^h slots, where a leaf (height 0) IS the stored value directly, and an internal node is a pair of two subtrees one level shorter. The key realisation that makes both 'update an existing element' and 'extend into brand new capacity' the SAME piece of code is that a subtree which has not been needed yet is simply represented by the value None, rather than being pre-built with placeholder leaves: a single recursive `_insert` routine, given a (possibly-None) subtree, either copies exactly the $O(1)$ nodes along the path to the target position while reusing every untouched sibling subtree unchanged (when the path already exists), or builds a fresh path of brand-new nodes as it goes (when it runs into a None along the way), and never needs to know in advance which of the two situations it is in. Growing past current capacity is then handled by wrapping the ENTIRE existing trie as the left child of a new, one-level-taller root whose right child starts out None, and then simply calling the same `_insert` on that.

Part B wraps a Ledger class around this: version 0 is always the empty vector, and every append or overwrite creates exactly one brand new version, published under the next available integer id. The performance point you are explicitly graded on here is that building a new PVector (the pure, functional work of Part A) touches only immutable, already-existing data and therefore needs NO lock at all; the only operation that actually touches SHARED, MUTABLE state — the dictionary of version id to PVector, and the counter handing out the next id — is the final, tiny 'publish this finished new version' step, and that is the only piece of work that must ever be done while holding your lock. A correct solution that instead holds one big lock around the entire append or set call from start to finish will still pass every correctness check in this problem, but it will needlessly force every reader and every writer in the whole system to queue up behind each other even when they are working with completely unrelated, already-finished versions that could safely be read concurrently, and the concurrency stress tests below are specifically built to reward the version of this class that does not do that.

Operations (sequential correctness tests)

Command	Effect
APPEND v x	Append x to version v, producing a new version. Appends the new version's id.
SET v i x	Set index i of version v to x, producing a new version. Appends the new version's id. Assumes $0 \leq i < \text{size}(v)$.
GET v i	Appends the element at index i of version v. Assumes $0 \leq i < \text{size}(v)$.
SIZE v	Appends the size of version v.

How this is graded

Sequential correctness (Part A and Part B's bookkeeping) is checked exactly like every other problem in this paper, via `visible_cases_*.txt` against `stdout`. The concurrency requirement on Ledger is checked separately, by two multithreaded stress programs that import your Ledger class directly: one hammers `append_to` from eight threads simultaneously and checks that every returned version id is unique and that no version numbers were skipped or duplicated (a race in your 'publish' step would show up here immediately); the other builds one long stable chain of versions, then spins up several reader threads continuously re-reading old, already-published versions while several writer threads are simultaneously creating brand-new versions elsewhere, and checks that not one single value in any already-published version is ever observed to have changed.

Assumptions and complexity

`pv_get`, `pv_set`, and `pv_append` must all be $O(\log n)$ worst-case, where n is the relevant vector's current size; `pv_size` must be $O(1)$. `append_to`, `set_in`, `get`, and `size` on Ledger must each hold the internal lock for $O(1)$ of the work, at most (the $O(\log n)$ tree work must happen outside the lock). A single sequential test may create on the order of 2×10^4 versions, including a deep chain of tens of thousands of sequential appends followed by thousands of independent branches off one early, shallow version. Concurrency tests run up to 8 threads simultaneously against one shared Ledger.

Requested files

- `student_code.py`
- `visible_cases_1.txt`
- `visible_cases_2.txt`